

CS 550 Operating Systems

Spring 2026

*More on process APIs
& fork memory semantics*

Process APIs

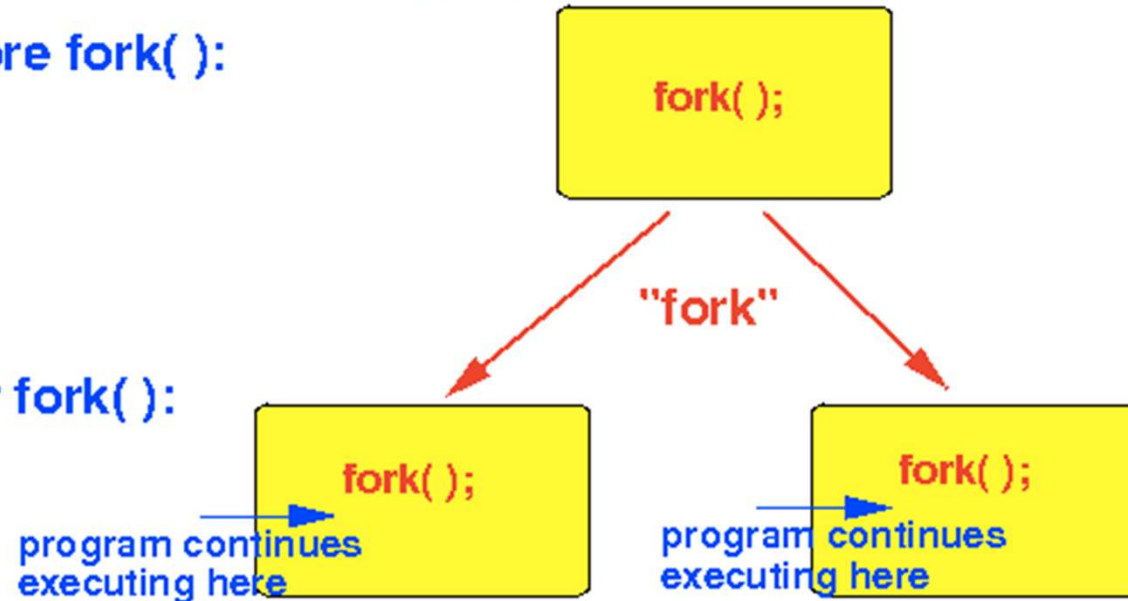
- `fork()`
 - Create a process. Every user process is created by its parent process, except the init process (PID=1).
- `exit(status)` and `_exit(status)`
 - Terminates the calling process, releasing all resources (memory, open file descriptors, and so on) used by the process to the OS.
- `wait(&status)`
 - Allows the parent process to wait one of its child processes terminates, and obtain the termination status.
- `exec functions`
 - Loads a new program into a process and executes it.

fork(): creating a new process

Running program:

Before fork():

After fork():



```
int main(int argc, char *argv[] )
{
    printf("Hello World !\n");

    fork( );

    printf("Good-bye World !\n");
}
```

```
Hello World !
Good-bye World !
Good-bye World !           // Printed twice !!!
```

fork(): creating a new process

```
5  int
6  main(int argc, char *argv[])
7  {
8      printf("hello world (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {          // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child (new process)
14         printf("hello, I am child (pid:%d)\n", (int) getpid());
15     } else {              // parent goes down this path (main)
16         printf("hello, I am parent of %d (pid:%d)\n",
17               rc, (int) getpid());
18     }
19     return 0;
20 }
```

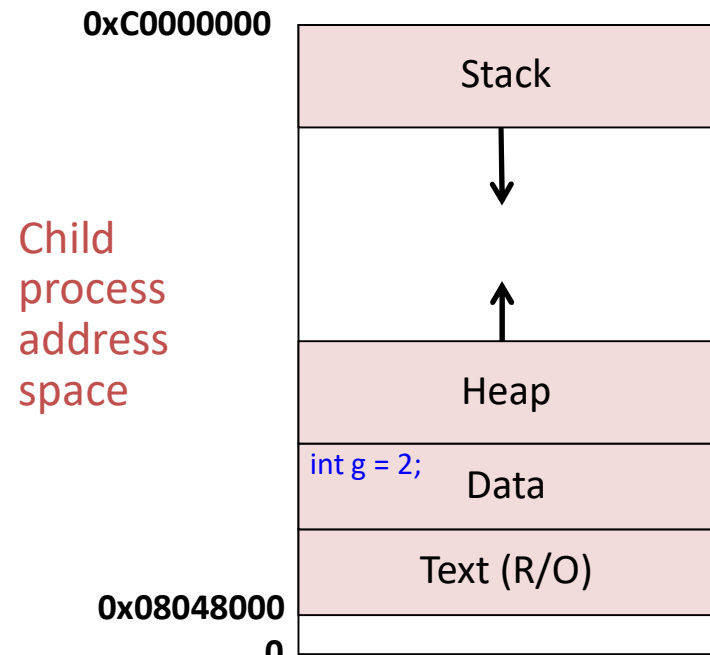
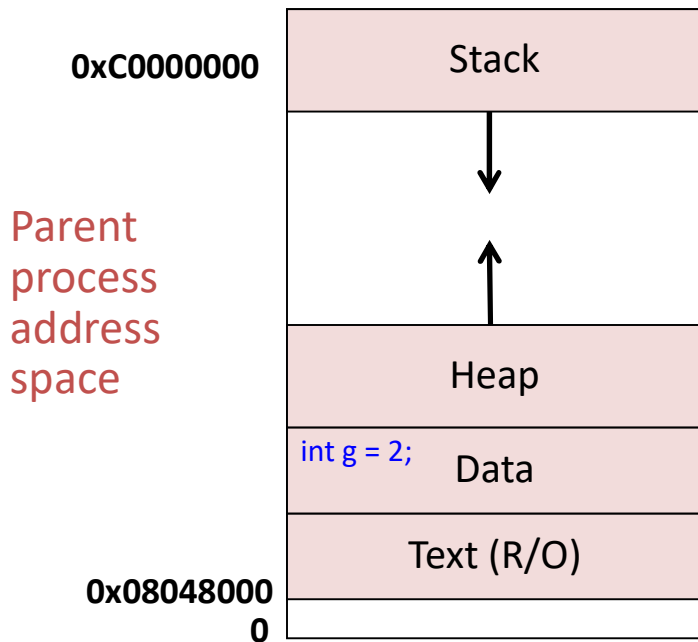
OSTEP Figure 5.1: **p1.c**: Calling **fork()**

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

Parent process and child process

- The child process has its own **private** address space, with the **initial content be exactly the same as the parent process**.
 - What does “private” suggest?



Parent process and child process

- When a parent process uses `fork()` to create a child process, the two processes have
 - the **same** program text (read only).
 - but **separate** copies of the data, stack, and heap segments.
- The child's stack, data, and heap segments are **initially exact duplicates of the corresponding parts the parent's memory**.
- After the `fork()`, **each process can modify the variables in its data, stack, and heap segments without affecting the other process**.

```

static int idata = 111;    /* Allocated in data segment */
int main(int argc, char *argv[])
{
    int istack = 222;      /* Allocated in stack segment */

    pid_t f_ret = fork();
    if (f_ret == 0)
    {
        idata *= 3;
        istack *= 3;
    }
    else
        sleep(1);

    printf("%s: idata=%d istack=%d\n", \
        (f_ret > 0) ? "Parent" : "Child", idata, istack);

    exit(0);
}

```

Suppose the fork() is invoked successfully,
what is the output of this program?

\$./t_fork

blank-1 : idata= blank-2 istack= blank-3
blank-4 : idata= blank-5 istack= blank-6

```

static int idata = 111;  /* Allocated in data segment */
int main(int argc, char *argv[])
{
    int istack = 222;    /* Allocated in stack segment */

    pid_t f_ret = fork();
    if (f_ret == 0)      /* Child Process: modify data */
    {
        idata *= 3;
        istack *= 3;
    }
    else                 /* Parent process: allow child to execute first*/
        sleep(1);

    /* Both parent and child come here */
    printf("%s: idata=%d istack=%d\n", \
        (f_ret > 0) ? "Parent" : "Child", idata, istack);

    exit(0);
}

```

\$./t_fork

Child: idata=333 istack=666

Parent: idata=111 istack=222

Typical uses of fork()

- Two typical uses of fork()
 - When a process wants to duplicate itself so that the parent and the child can each execute different sections of code at the same time (example: network daemons/servers).
 - When a process wants to execute a different program.
 - In this case, the child does an `exec` function right after it returns from the `fork()`.

`exit()` and `_exit()`: terminating a process

- A process may terminate in two general ways:
 - One is abnormal termination, which is caused by the delivery of a signal whose default action is to terminate the process.
 - The other way is that a process can terminate normally, using the `exit()` or `_exit()` function.

exit() and _exit(): terminating a process

```
#include <unistd.h>

void _exit(int status);
```

- The status argument given to _exit() defines the termination status of the process, which is available to the parent of this process when it calls wait().
- By convention, a termination status of 0 indicates a process completed successfully.
- How to interpret non-zero termination status is up to the application.

`wait()`: collecting child process termination status

```
#include <sys/wait.h>
```

```
pid_t wait(int *status);
```

Returns process ID of terminated child, or -1 on error

- It waits for one of the children of the calling process to terminate and returns the termination status of that child (i.e., the status that the child process passed to `exit()` or `_exit()`) in the buffer pointed to by *status* (if *status* is not a NULL pointer).
- Returns the process ID of the terminated child, or -1 on error (e.g., no child process or all child processes have been waited for).

```

int
main(int argc, char *argv[])
{
1   printf("hello world (pid:%d)\n", (int) getpid());
2   int rc = fork();
3   if (rc < 0) {           // fork failed; exit
4       fprintf(stderr, "fork failed\n");
5       exit(1);
6   } else if (rc == 0) { // child (new process)
7       printf("hello, I am child (pid:%d)\n", (int) getpid())
8   } else {                // parent goes down this path (main)
9       int wc = wait(NULL);
10      printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
              rc, wc, (int) getpid());
    }
    return 0;
}

```

OSTEP Figure 5.2: **p2.c: Calling `fork()` And `wait()`**

```

prompt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (wc:29267) (pid:29266)
prompt>

```

`exec`: executing a new program

- The `exec` functions are useful when you want to run a program that is different from the calling program.
 - For example, calling `fork()` in `p2.c` is only useful if you want to keep running copies of the same program. However, often you want to run a different program; `exec()` does just that.

exec: executing a new program

```
7  int
8  main(int argc, char *argv[])
9  {
10     printf("hello world (pid:%d)\n", (int) getpid());
11     int rc = fork();
12     if (rc < 0) {           // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child (new process)
16         printf("hello, I am child (pid:%d)\n", (int) getpid());
17         char *myargs[3];
18         myargs[0] = strdup("wc"); // program: "wc" (word count)
19         myargs[1] = strdup("p3.c"); // argument: file to count
20         myargs[2] = NULL;          // marks end of array
21         execvp(myargs[0], myargs); // runs word count
22         printf("this shouldn't print out");
23     } else {                 // parent goes down this path (main)
24         int wc = wait(NULL);
25         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
26               rc, wc, (int) getpid());
27     }
28     return 0;
29 }
```

Figure 5.3: **p3.c**: Calling **fork()**, **wait()**, And **exec()**

exec: executing a new program

- The exec functions load a new program into a process's memory. The old program is completely replaced by the new program in the process.
- The text segment is replaced by the new program's .text section.
- The process's data, stack and heap segments are reinitialized according to the new program.

Discussion

- You write a Linux program, and in the main function your code first calls `exec()` and then `fork()` like the following. Does this work (i.e., is the `fork()` function going to be invoked)? Why?

```
main (...) {  
    ...  
    exec (cmd, args);  
    fork();  
    ...  
}
```

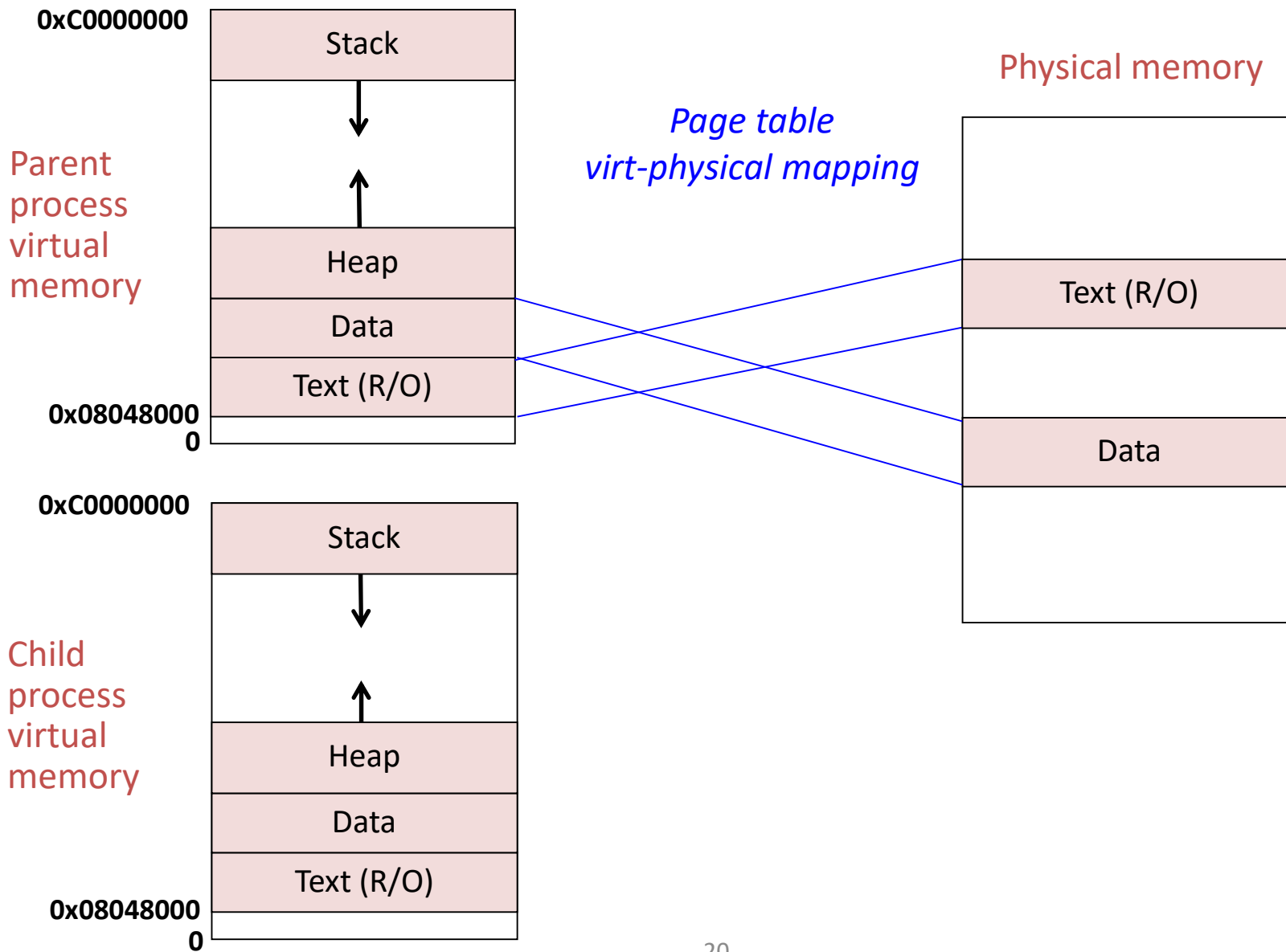
Recap: fork() memory semantics

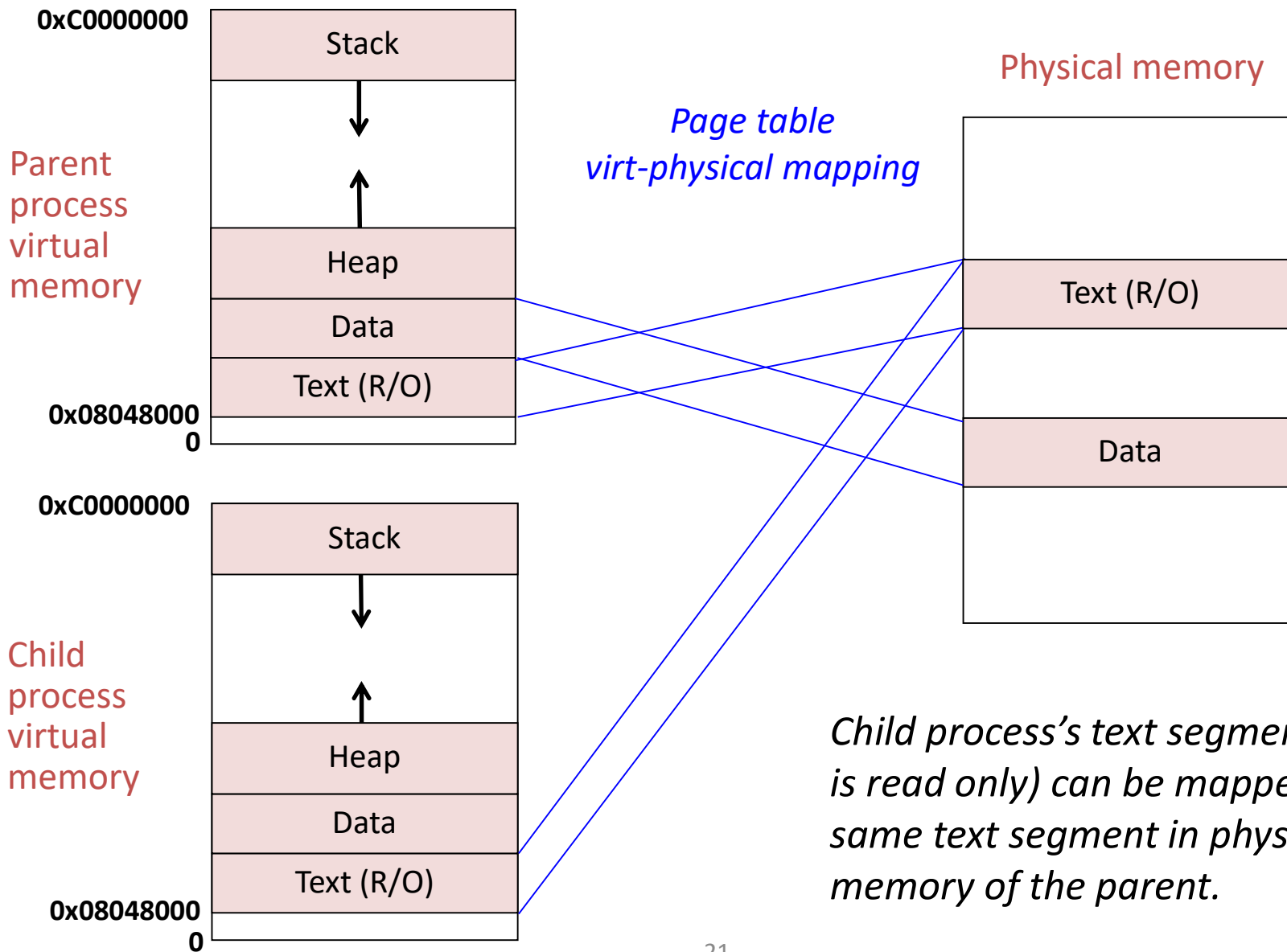
- When a parent process uses fork() to create a child process, the two processes have
 - the **same** program text (read only).
 - but **separate** copies of the data, stack, and heap segments.
- The child's stack, data, and heap segments are **initially exact duplicates of the corresponding parts the parent's memory**.
- After the fork(), **each process can modify the variables in its data, stack, and heap segments without affecting the other process**.

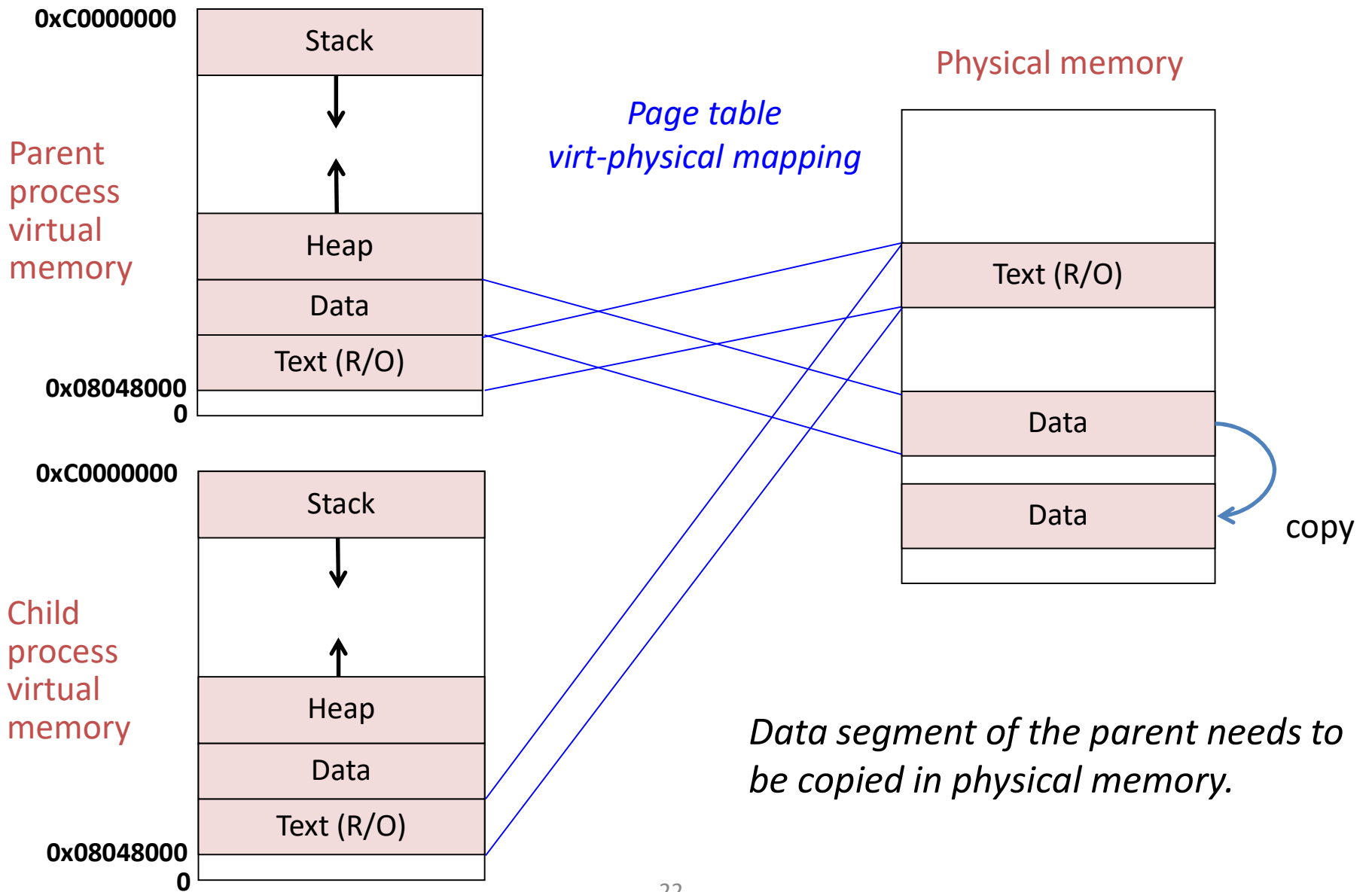
How does OS support this memory semantics?

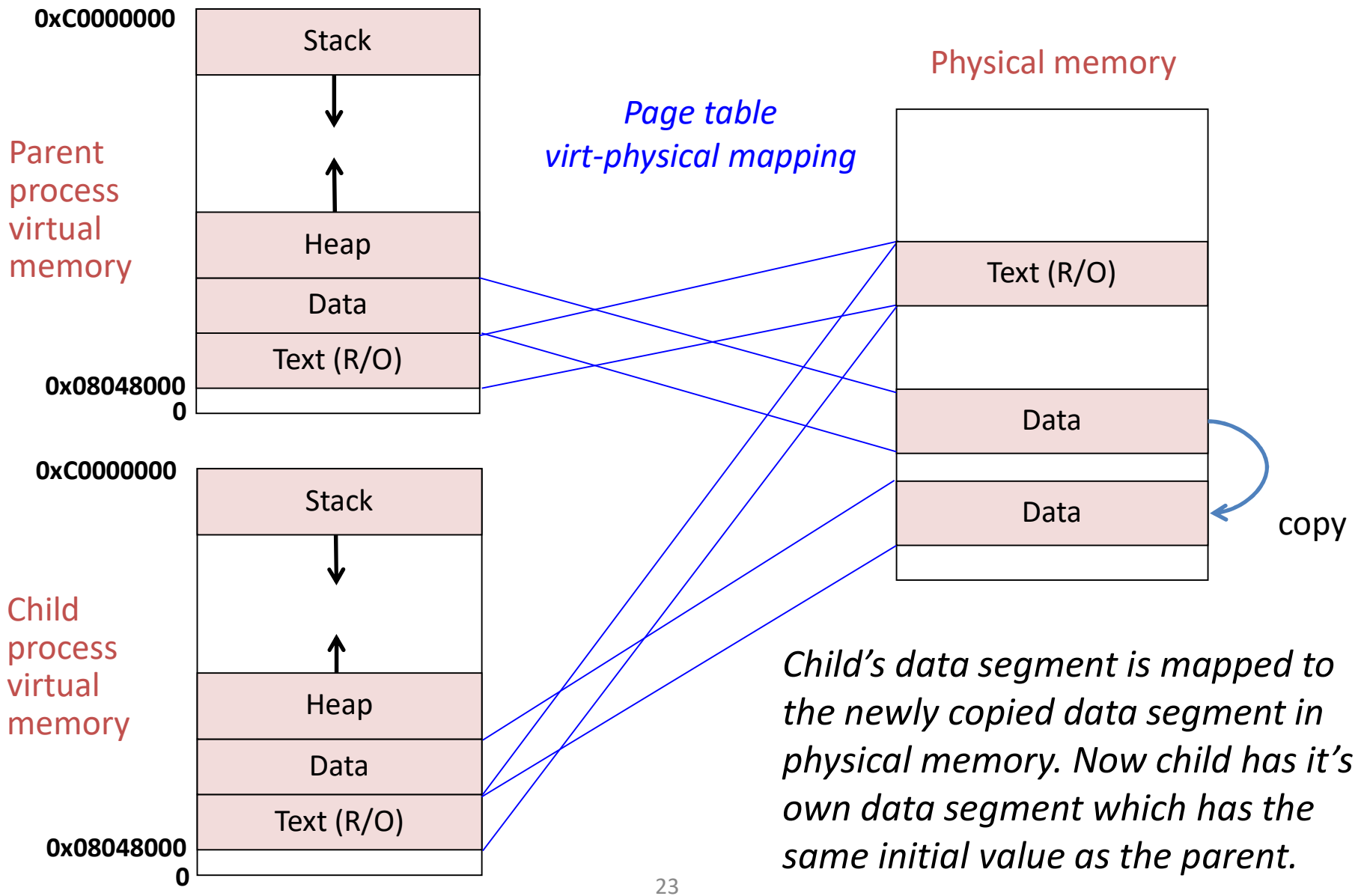
OS support for memory semantics of `fork()`

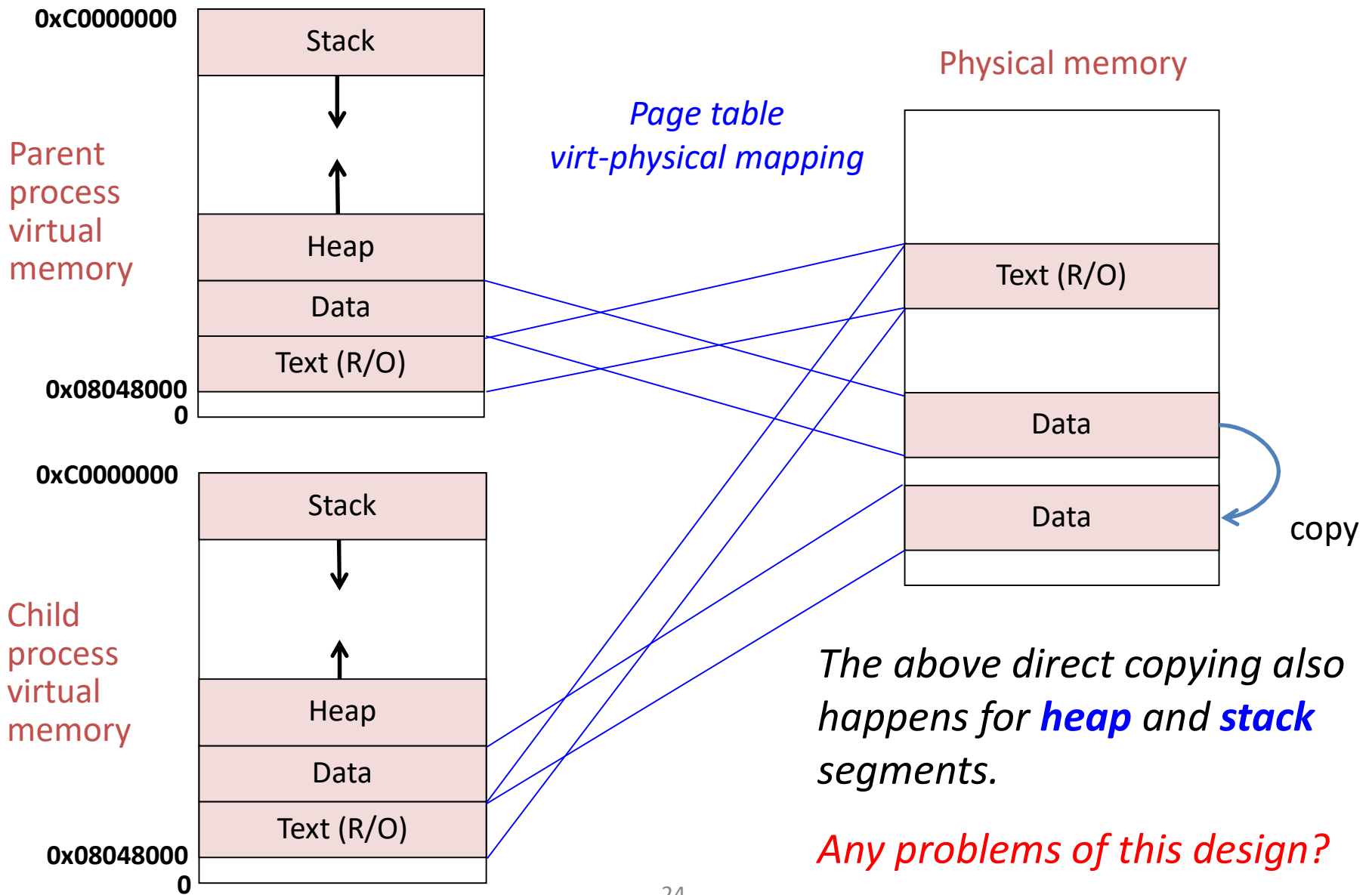
- Straightforward solution: when forking, create copies of the parent's data, stack, and heap segments in physical memory for the child process.
 - This was actually the implementation used by some early Unix systems.
 - It look like this ...











Memory semantics of `fork()`

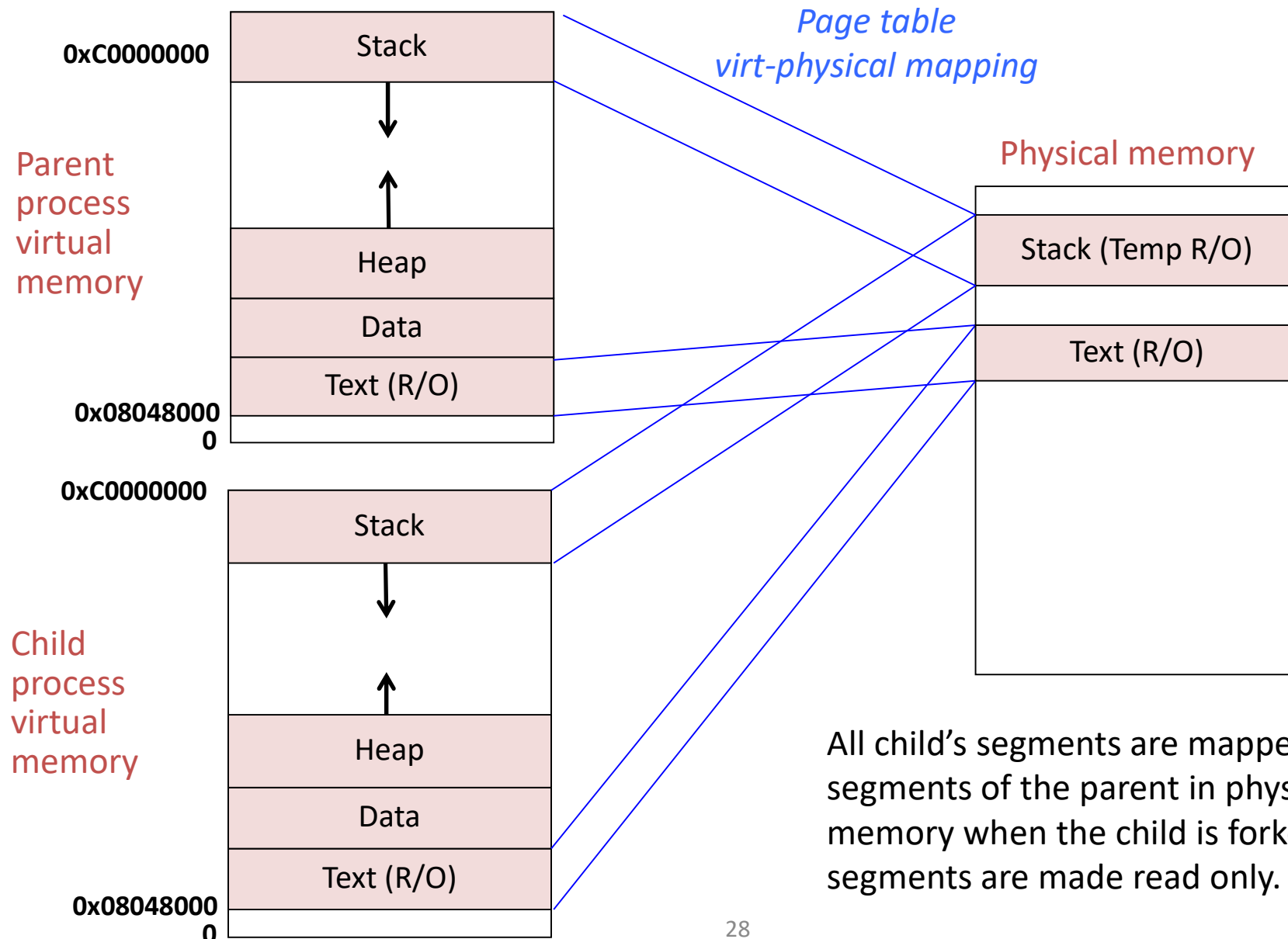
- Problems of direct copying parent's memory pages when forking.
 - Slow process creation - the child process won't be ready until all the memory copying is done.
 - Waste of resources and time
 - Many resources could be shared between parent and child.
 - `fork()` is usually immediately followed by an `exec` function to run a different program, which causes a replacement of the child process text segment, and reinitialization of the data, stack and heap segments, making the direct copying when forking a waste.

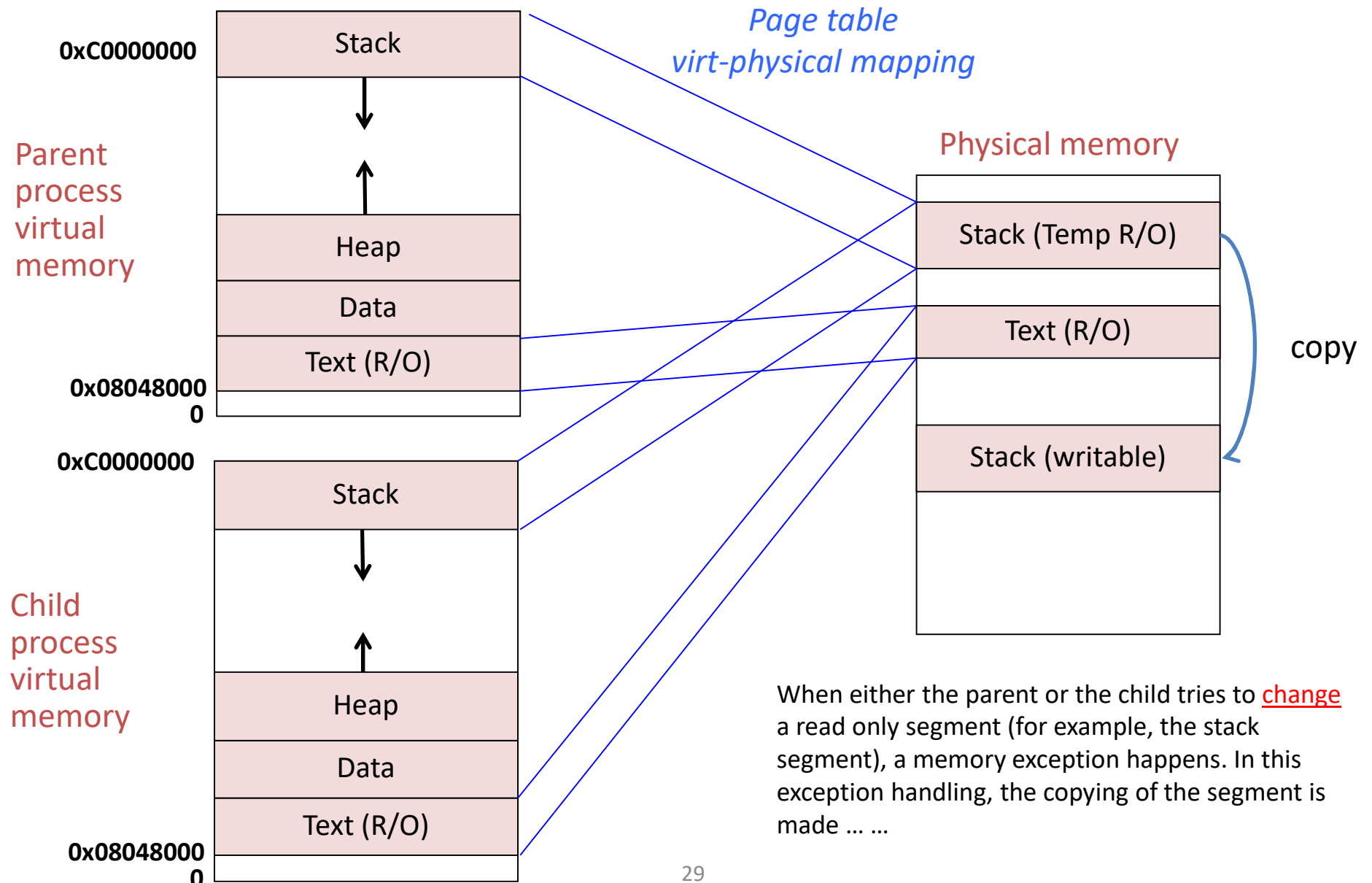
Copy-on-write for `fork()`

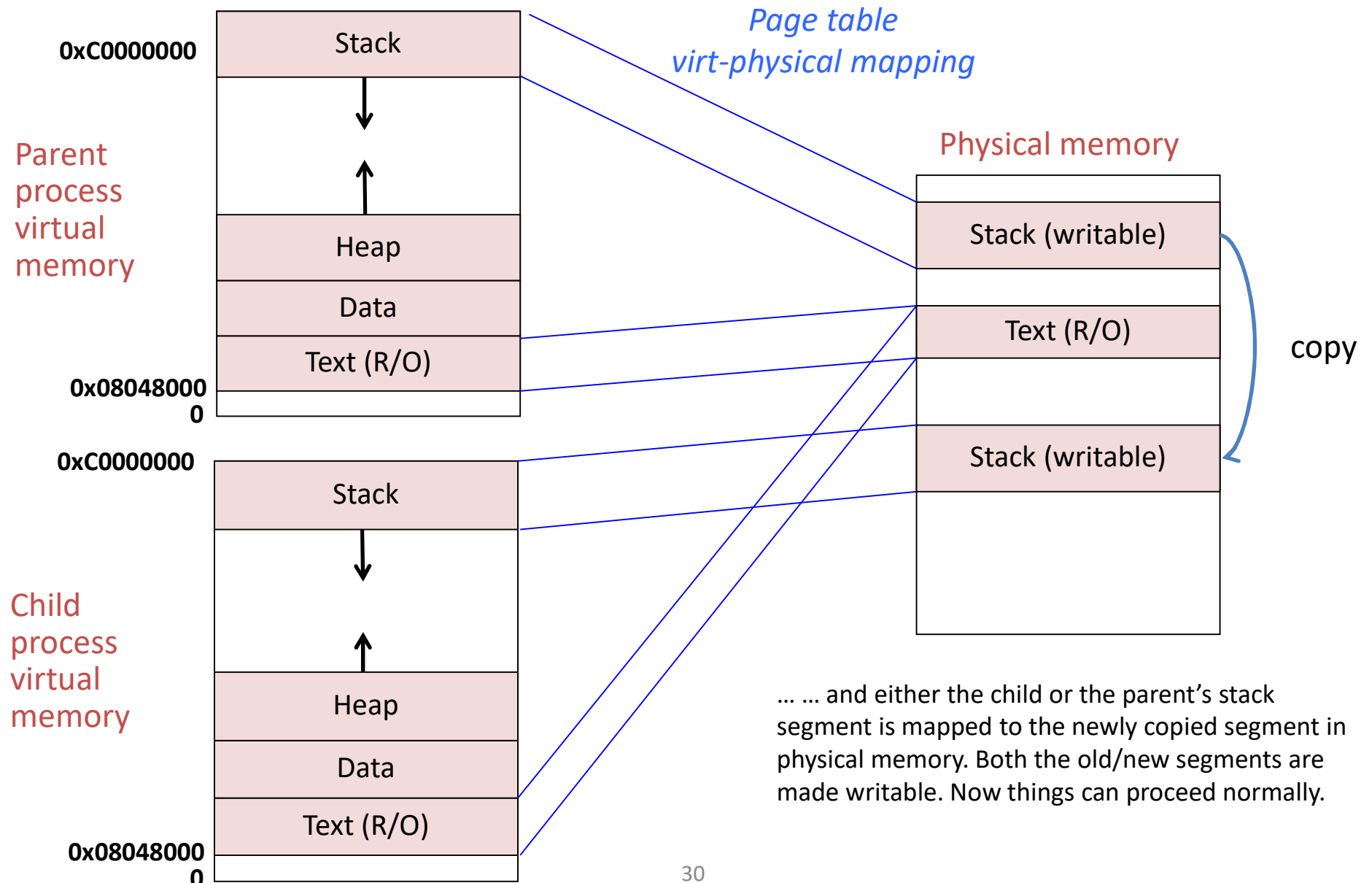
- Most modern Unix implementations apply the *copy-on-write* (*COW*) technique for data, stack, and heap segments.
 - When `fork()` is performed, the child's page table entries for data, stack, and heap segments have the same mappings as those in the parent process's page table (meaning address spaces of parent and child processes share the same physical memory).
 - All the pages related to the data, stack, and heap segments are marked as *read-only* (R/O).

Copy-on-write for `fork()`

- Any attempt by either the child or the parent trying to modify the R/O pages will generate a page fault (i.e., a type of exception related to memory).
- When handling the page fault, the kernel duplicates the about-to-be-modified page, and assigns it to the faulting process, and updates the page table of the faulting process accordingly.
- Kernel marks the original and the duplicated pages as writable.
- Now the parent and the child can modify their own copies of pages without affecting each other's pages.







Copy-on-write for `fork()`

- What is the overhead incurred by `fork()` with COW?
 - The only overhead is the duplication of the parent's page tables and the creation of a unique process descriptor (`struct task_struct` in Linux kernel) for the child.
 - In the common case that a process executes a new executable binary immediately after forking, this optimization prevents the wasted copying of large amounts of data.
 - This is an important optimization because the Unix philosophy encourages quick process execution.

Race conditions after `fork()`

- After `fork()`, either the parent or the child can be scheduled to run.
 - A test of executing `fork()` multiple times in Linux 2.2.19: 99.97% of a million executions have the parent first to run.
 - In Linux 2.4, change was made to have the child to run first most of the time. But this change was later dropped from the 2.4 kernel series.
 - In Linux 2.6, the change to have the child run first was adopted.

Race conditions after `fork()`

- What is the reasoning of the two policies?
 - Child process to run first policy.
 - If parent is first scheduled to run and parent makes changes to data, stack or heap segments, a copy of to-be-changed pages will be duplicated.
 - If the child perform an `exec` right after it gains the CPU, the previous duplication becomes be a waste.
 - Therefore by schedule the child first can avoid unnecessary copying.
 - Parent process to run first policy.
 - Cache locality: after a `fork()`, the parent's state is already active in the CPU and its memory-management information is already cached in the hardware memory management unit's translation look-aside buffer (TLB).

```
...  
if (fork () > 0) {  
    // Parent  
    ... // update mem  
}  
else {  
    // Child  
    exec();  
}  
...
```